SPE Amplifier Remote Control

User Guide

For SPE Expert 1.3K-FA / 1.5K-FA / 2K-FA HF Amplifiers

Original script by **OH2GEK**
Modernized and extended by **VU2CPL**

Item	Value
Version	3.0.2 (refuses cross-origin WebSocket connections)
Platform	Raspberry Pi / Linux / macOS
Interface	Web browser + MacExpert native
Connection	USB / RS-232 Serial
Protocol	SPE App Programmer's Guide Rev 1.1

Table of Contents

1. Introduction
2. Requirements
3. Installation
4. Configuration
5. Starting the Server
6. Web Interface Guide
7. Power On / Off Control
8. Controls Reference
9. RCU (Remote Control Unit) Mode
10. Architecture
11. SPE Serial Protocol Reference
12. Running as a System Service
13. WebSocket API
14. Troubleshooting

1. Introduction

SPE Remote Control is a modern Python 3 application that allows you to monitor and control SPE Expert HF amplifiers remotely from any web browser or from the MacExpert native macOS companion app. It communicates with the amplifier via USB or RS-232 and serves a real-time web dashboard plus a binary RCU LCD mirror over a single WebSocket.

Key Features:

- **Power On/Off** - remote power control via DTR line (on) and serial command 0x0A (off).
- **Full SPE protocol** - all commands from the official Programmer's Guide plus undocumented RCU commands.
- **RCU LCD mirror** - live front-panel display streamed as binary frames for pixel-accurate rendering.
- **Self-contained** - single process serves both the WebSocket API and web UI.
- **Multi-client** - multiple browsers and devices simultaneously.
- **Mixed-client broadcast** - text JSON for browsers, binary frames for RCU clients, same socket.
- **Threaded reader + async writer** - survives USB-serial poll glitches that crash serial_asyncio.
- **Graceful shutdown** - SIGINT/SIGTERM handler cancels tasks and closes the port cleanly.
- **Auto-reconnect** - WebSocket and serial both reconnect automatically on failure.
- **Configurable** - YAML configuration file for serial port, baud, polling, heartbeat.

2. Requirements

Hardware:

- Raspberry Pi (any model - Pi 2, 3, 4, 5, or Zero 2 W) or any Linux/macOS computer
- SPE Expert amplifier (1.3K-FA, 1.5K-FA, 2K-FA, or compatible model)
- USB cable (with FTDI adapter) or RS-232 serial connection to the amplifier
- Network connection (Ethernet or Wi-Fi)

Software:

- Python 3.9 or newer (included in Raspberry Pi OS)
- python3-venv package (for virtual environment)
- Git (for cloning the repository)

Python Dependencies (installed automatically):

Package	Version	Purpose
tornado	>= 6.0	Web server and WebSocket framework
pyserial	>= 3.5	Serial port (blocking reads on reader thread)
pyyaml	>= 6.0	YAML configuration file parsing

3. Installation

Step 1: Clone the Repository

```
git clone https://github.com/vu2cpl/spe-remote.git
cd spe-remote
```

Step 2: Run Setup

```
./setup.sh
```

The setup script creates a Python virtual environment and installs all dependencies. On Raspberry Pi OS, it will also install python3-venv if needed.

Step 3: Add User to dialout Group

```
sudo usermod -aG dialout $USER
```

Log out and back in for the change to take effect.

4. Configuration

All settings are in **config.yaml** in the project root.

```
serial:
  port: /dev/serial/by-id/usb-FTDI_FT232R_USB_UART_...
  baudrate: 115200
  timeout: 1.0

server:
  port: 8888
  host: "0.0.0.0"

polling:
  tx_interval: 0.2
  idle_interval: 1.0
  heartbeat: 15

logging:
  level: INFO
```

Configuration Options:

Setting	Default	Description
serial.port	/dev/ttyUSB0	Serial port path to amplifier
serial.baudrate	115200	Serial baud rate
server.port	8888	HTTP/WebSocket listen port
server.host	0.0.0.0	Listen address (0.0.0.0 = all ifaces)
polling.tx_interval	0.2	Poll interval during TX (sec)
polling.idle_interval	1.0	Poll interval during RX/Standby
polling.heartbeat	15	Force state broadcast interval
updates.check	true	Daily GitHub new-release check (banner)
logging.level	INFO	DEBUG / INFO / WARNING / ERROR

Finding Your Serial Port:

```
ls /dev/serial/by-id/
dmesg | grep ttyUSB
```

Tip: Use /dev/serial/by-id/... paths instead of /dev/ttyUSB0. The by-id paths are stable across reboots.

5. Starting the Server

Interactive (foreground):

```
./run.sh
```

Detached with log to nohup.out:

```
nohup ./run.sh &
```

Or start directly with the venv Python:

```
venv/bin/python server.py
```

With a custom config — pass the path to your YAML config file as the first argument. The file must exist; if it doesn't, the server exits with an error rather than silently falling back to defaults.

```
venv/bin/python server.py YOUR-CONFIG.yaml
```

On successful startup you will see:

```
[INFO] spe: Server listening on http://0.0.0.0:8888/
[INFO] spe: Serial port: /dev/ttyUSB0 @ 115200 baud
[INFO] spe.serial_handler: Connecting...
[INFO] spe.serial_handler: Serial connected
```

Stop with Ctrl+C - the shutdown handler cancels all tasks, sends RCU OFF to the amp, and closes the port cleanly.

6. Web Interface Guide

The bundled browser dashboard at http://<pi>:8888/ decodes text JSON state messages and ignores the binary RCU frames. For the full LCD mirror, use the MacExpert native client.

Connection Indicator

The green/red dot at the top shows WebSocket connection status. On disconnect, the client auto-reconnects with exponential backoff.

Power Output Display

The large power bar shows current output power in watts (0-1500W). The gradient color changes from green through cyan and yellow to red.

Gauges

Gauge	Range	Warning Zone
SWR	1:1 to 1:3.5	Above 1:2.0 (orange), above 1:2.8 (red)
Drain Current	0 - 60 A	Above 42 A (orange), above 51 A (red)
PA Temperature	0 - 80 C	Above 60 C (orange), above 64 C (red)
Voltage	40 - 60 V	Below 44 V or above 55 V (warning)

Status Chips

Status row shows TX/RX (red pulse during TX, green during RX), current band, antenna number, input number, and power level.

Alert Bar

Warning and error codes from the amplifier show here in orange (warning) or red (error). Power action results show green (ok) or red (error), auto-clearing after 4 seconds.

7. Power On / Off Control

The web interface includes Power ON and Power OFF buttons. These use different mechanisms as defined by the SPE protocol.

Power ON (DTR hardware toggle)

There is no serial data command for power on. Power ON is performed by toggling the DTR line on the USB-serial adapter. The sequence is based on the original power_spe_on.py script by OH2GEK:

```
DTR=1 -> DTR=0 -> RTS=1 -> wait 1s -> DTR=1 -> RTS=0
```

After the sequence, the amplifier takes 3 to 4.5 seconds to start. While DTR is held high, the amp shows 'POWER SWITCH HELD BY REMOTE' and the front panel switch is overridden.

Power OFF (serial command 0x0A)

Power OFF uses the official SPE serial command **SWITCH OFF (0x0A)** from the Application Programmer's Guide - equivalent to pressing the front-panel OFF button.

```
Packet: 0x55 0x55 0x55 0x01 0x0A 0x0A
```

Action	Method	Mechanism
Power ON	DTR hardware toggle	USB-serial DTR/RTS line sequence
Power OFF	Serial command 0x0A	SWITCH OFF per SPE protocol

Safety:

Both Power ON and Power OFF require a browser confirmation dialog. The buttons show a shimmer animation while processing and display a success or error result in the alert bar.

8. Controls Reference

All commands below are sent to the amp via WebSocket text messages. Clients just send the bare command name (e.g. 'oper'); the server wraps it in the SPE packet format.

Command	Hex	Description
`power_on`	DTR	Power ON via DTR toggle. Confirmation required.
`power_off`	0x0A	SWITCH OFF - power down. Confirmation required.
`oper`	0x0D	Toggle Operate / Standby.
`antenna`	0x04	Cycle TX antenna (ANT 1, ANT 2, ...).
`tune`	0x09	Start ATU tuning cycle.
`input`	0x01	Toggle input (IN 1 / IN 2).
`power_level`	0x0B	Cycle Low / Mid / High power.
`band_up`	0x03	Step up one band.
`band_dn`	0x02	Step down one band.
`l_plus / l_minus`	0x06 / 0x05	ATU inductance + / -
`c_plus / c_minus`	0x08 / 0x07	ATU capacitance + / -
`left / right / set`	0x0F / 0x10 / 0x11	Menu navigation
`display`	0x0C	Toggle display.
`cat`	0x0E	CAT mode.
`rcu_on / rcu_off`	0x80 / 0x81	Enable / disable LCD mirror stream.
`backlight_on / off`	0x82 / 0x83	Backlight control.

Note: All connected clients can send commands. In a multi-client setup, coordinate to avoid conflicts.

9. RCU (Remote Control Unit) Mode

RCU is a streaming mode that mirrors the amplifier front-panel LCD over the serial link. When enabled, the amp emits binary frames (marker 0x6A) every time the display changes, alongside regular CSV status polling.

How the Server Uses RCU

- On serial connect, the handler sends CMD_REQUEST (status) followed by CMD_RCU_ON.
- A background task cycles RCU_OFF -> RCU_ON every 500 ms to keep the stream alive (the amp sometimes stops emitting after a long quiet period).
- A quiet-flush task force-terminates any half-received RCU frame after 300 ms of silence - so static screens still emit their final frame.
- Incoming RCU payloads are passed to the on_rcu_frame callback, which broadcasts them as binary WebSocket messages.

Why Two Frame Types?

CSV status - machine-readable data (power, SWR, temps, warnings). Parsed into JSON, sent as WebSocket text. This is what the browser dashboard reads.

RCU frames - pixel-level view of the LCD. Lets a native client render an exact replica of the front panel, including menu screens and settings that aren't in the CSV.

Client Matrix

Client	Platform	Uses CSV	Uses RCU
Web dashboard	Browser	Yes	No (drops binary)
MacExpert	macOS	Yes	Yes

Command Contract with MacExpert

The COMMANDS dict keys in spe/protocol.py are the canonical WebSocket command names. MacExpert's SPEProtocol.swift has a matching wsCommandName enum so both clients drive the amp identically. When adding a new command, update both sides.

10. Architecture

Threading Model (serial_handler.py)

The serial handler uses a hybrid thread + asyncio model instead of pyserial-asyncio:

```
Asyncio event loop (main thread)
  - _poll_loop: periodic status requests
  - _command_loop: drains queue, writes to port
  - _rcu_tick_loop: keeps RCU stream alive
  - _quiet_flush_loop: flushes stalled RCU frames
  - _connection_watchdog
  - Frame parser: drains receive buffer

Daemon thread
  - Blocking serial.Serial.read() loop
  - Pushes chunks via call_soon_threadsafe()

Synchronization
  - threading.Lock on all writes (_safe_write)
  - threading.Event to signal reader shutdown
```

Why not serial_asyncio?

Its internal _read_ready callback raises SerialException("readiness to read but returned no data") on Linux USB-serial adapters under moderate traffic. That bounces the port and breaks the RCU stream. The blocking serial.read() path used here never hits that bug, so the port stays up even under full RCU load.

Key Constants

Constant	Value	Purpose
_RCU_TICK_INTERVAL	0.5 s	RCU OFF->ON cycle cadence
_RCU_OFF_ON_GAP	0.05 s	Gap between RCU OFF and ON
_RCU_QUIET_FLUSH	0.3 s	Force-flush stuck RCU frames
_READ_TIMEOUT	0.1 s	Blocking serial read timeout
_MAX_BUFFER	4096 bytes	Receive buffer cap

Lifecycle

- server.py creates SerialHandler, PowerController, and the Tornado app.
- Installs SIGINT/SIGTERM handlers that schedule serial_handler.stop() on the asyncio loop, then stop both Tornado and asyncio loops.
- serial_handler.start() loops: open port -> spawn reader thread -> run the five asyncio tasks via asyncio.wait(FIRST_COMPLETED) -> tear down on any task exit -> reconnect after 3 s.
- On stop(): sets _stop_reader event, sends RCU_OFF to the amp, closes the port, drops queued commands. The reader thread exits once its port handle is closed.

11. SPE Serial Protocol Reference

Based on the **SPE Application Programmer's Guide Rev 1.1** for Expert 1.3K-FA / 1.5K-FA / 2K-FA. Communication is asynchronous, 8N1, up to 115200 baud (auto-adapts).

Packet Format

```
0x55 0x55 0x55 [CNT] [DATA...] [CHK] (host -> amp)
0xAA 0xAA 0xAA [CNT/TYPE] [DATA...] ... (amp -> host)

For single-byte commands: CNT=0x01, CHK=DATA
```

Complete Command Set

Hex	Command	Description
0x01	INPUT	Toggle input port
0x02	BAND -	Band down
0x03	BAND +	Band up
0x04	ANTENNA	Cycle TX antenna
0x05	L-	ATU inductance minus
0x06	L+	ATU inductance plus
0x07	C-	ATU capacitance minus
0x08	C+	ATU capacitance plus
0x09	TUNE	Start ATU tuning
0x0A	SWITCH OFF	Power OFF the amplifier
0x0B	POWER	Toggle power level (L/M/H)
0x0C	DISPLAY	Display toggle
0x0D	OPERATE	Toggle Operate/Standby
0x0E	CAT	CAT mode
0x0F	LEFT ARROW	Menu navigation left
0x10	RIGHT ARROW	Menu navigation right
0x11	SET	Menu enter / set
0x80	RCU ON	Enable LCD mirror stream (undocumented)
0x81	RCU OFF	Disable LCD mirror stream (undocumented)
0x82	BACKLIGHT ON	Turn display backlight on
0x83	BACKLIGHT OFF	Turn display backlight off
0x90	STATUS	Request status string

Response Frame Types

Marker	Type	Length	Description
0x43	CSV status	67 + CRC + CRLF	ASCII comma-separated status

0x6A	RCU frame	Variable	Binary LCD display payload

Status String Fields

Field	Len	Contents
ID	3	20K (2K-FA) or 13K (1.3K-FA)
Stby/Oper	1	S or O
RX/TX	1	R or T
Memory	1	A, B, or x
Input	1	1 or 2
Band	2	00 (160m) to 11 (4m)
TX Ant+ATU	2	0-6, suffix t/b/a
RX Ant	2	Antenna number or 0r
Power Level	1	L, M, or H
Output Power	4	Watts (0000 on RX)
SWR ATU	5	VSWR before ATU
SWR ANT	5	VSWR at antenna
V PA	4	Supply voltage
I PA	4	Drain current
Temp Upper	3	Heatsink temp
Temp Lower	3	Lower heatsink (2K-FA)
Temp Combiner	3	Combiner temp (2K-FA)
Warnings	1	Code, N = none
Alarms	1	Code, N = none

Warning Codes:

M=Alarm, **A**=No antenna, **S**=SWR, **B**=No band, **P**=Power limit, **O**=Overheat, **Y**=ATU N/A, **W**=Tune no power, **K**=ATU bypass, **R**=Remote hold, **T**=Combiner heat, **C**=Combiner fault, **N**=None

Alarm Codes:

S=SWR limit, **A**=Amp protection, **D**=Overdrive, **H**=Excess heat, **C**=Combiner fault, **N**=None

12. Running as a System Service

To start the server automatically on boot:

Step 1: Create the Service File

```
sudo nano /etc/systemd/system/spe-remote.service
```

Paste:

```
[Unit]
Description=SPE Amplifier Remote Control
After=network.target

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/spe-remote
ExecStart=/home/pi/spe-remote/venv/bin/python server.py
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Adjust User and WorkingDirectory to your actual install path.

Step 2: Enable and Start

```
sudo systemctl daemon-reload
sudo systemctl enable spe-remote
sudo systemctl start spe-remote
```

Step 3: Check Status

```
sudo systemctl status spe-remote
journalctl -u spe-remote -f
```

13. WebSocket API

Connect to **ws://<host>:8888/ws**. The socket carries three kinds of server-to-client messages: JSON state updates (text), JSON power-action results (text), and raw RCU LCD frames (binary). Clients that don't care about RCU should ignore binary messages.

Server -> Client: Amplifier State (text)

```
{
  "op_status": "Oper",
  "tx_status": "TX",
  "input": "1",
  "band": "80m",
  "tx_antenna": "1",
  "p_level": "10",
  "p_out": "1353",
  "swr": "1.54",
  "aswr": "1.12",
  "voltage": "54.6",
  "drain": "27.3",
  "pa_temp": "26",
  "warnings": "",
  "error": ""
}
```

Server -> Client: Power Action Result (text)

```
{
  "power_result": "power_on",
  "status": "ok"
}
```

Server -> Client: RCU Frame (binary)

Raw bytes after the AA AA AA 6A sync+marker. Decoding is client-specific; see MacExpert's RCUFrameDecoder.swift for reference.

Client -> Server: Commands (text)

Bare command name, e.g. 'oper'. Server routes power_on to PowerController.power_on() (DTR toggle), power_off to PowerController.power_off() (0x0A), everything else to SerialHandler.send_command().

JavaScript Client Example

```
const ws = new WebSocket("ws://<pi>:8888/ws");

ws.onmessage = (evt) => {
  if (typeof evt.data === "string") {
    const msg = JSON.parse(evt.data);
    if (msg.power_result) { /* power result */ }
    else { /* state update */ }
  } else {
    // Binary = RCU LCD frame
    evt.data.arrayBuffer().then(buf => renderRCU(new Uint8Array(buf)));
  }
};
```

```
ws.send("oper");
ws.send("band_up");
ws.send("power_off");
```

14. Troubleshooting

Problem: Serial error: No such file or directory

Solution: The serial port path in config.yaml does not exist. Check the cable is plugged in and verify with: ls /dev/serial/by-id/

Problem: Permission denied on serial port

Solution: Add your user to the dialout group: sudo usermod -aG dialout $USER, then log out and back in.

Problem: Web page not loading

Solution: Check the server is running and the port isn't blocked by a firewall. Try: sudo ufw allow 8888/tcp

Problem: Gauges not updating

Solution: Open browser dev tools (F12) and check for WebSocket errors. Verify the server IP and port are correct.

Problem: Multiple /dev/ttyUSB devices

Solution: Use /dev/serial/by-id/ which is unique per adapter.

Problem: Power ON not working

Solution: Verify your FTDI USB-serial adapter supports DTR line control. Check dmesg for adapter recognition. DTR toggle requires hardware support.

Problem: 'POWER SWITCH HELD BY REMOTE' warning

Solution: Normal when DTR is held high after a remote power on. The front panel switch is overridden while the remote has control.

Problem: 'Suppressed spurious USB-serial poll glitch' in logs

Solution: Harmless. The Linux USB-serial stack sometimes lies about poll readiness - the reader thread handles it without dropping the port.

Problem: RCU frames never arrive

Solution: Check the companion client actually reads binary WebSocket messages - the browser dashboard drops them silently by design.

Problem: Server doesn't exit on Ctrl+C

Solution: Shouldn't happen with the new shutdown handler. If it does, check for a hung serial read and report the issue.

Problem: MacExpert can't send commands

Solution: Verify the wsCommandName enum in Swift matches the COMMANDS keys in spe/protocol.py. Both sides must be kept in sync when adding commands.

Problem: Server crashes on startup

Solution: Check the log for errors. Common causes: wrong Python version (need 3.9+), missing dependencies (re-run ./setup.sh), or serial port in use by another program.

Problem: High CPU usage

Solution: Increase polling.idle_interval in config.yaml to reduce poll frequency when idle. Default 1.0 s is fine for most setups.

For additional help, open an issue on the GitHub repository:
https://github.com/vu2cpl/spe-remote

73 de OH2GEK & VU2CPL